

Hoja de Trabajo 1: Redes Neuronales

Deep Learning y Sistemas Inteligentes - CC3092

José Antonio Mérida Castejón

12 de julio de 2026

Task 1

Responda de forma clara:

- a. *Explique con sus propias palabras por que una red neuronal compuesta únicamente de transformaciones lineales, sin funciones de activación no lineales, tiene exactamente la misma capacidad expresiva que un modelo lineal de una sola capa. Su explicación debe incluir el argumento algebraico central.*

En términos resumidos, la composición de capas lineales resulta en otra capa lineal. Esto quiere decir que no importa cuantas veces multipliquemos la entrada por una matriz de pesos, el resultado seguirá siendo el mismo que multiplicarlo por una única matriz. Además, sin importar cuantos vectores de *bias* sumemos seguiremos obteniendo otro vector de *bias*. Para una demostración más formal consideremos lo siguiente:

Una capa lineal aplica a la entrada $x \in \mathbb{R}^n$ una transformación de la forma

$$f_1(x) = W_1x + b_1,$$

donde $W_1 \in \mathbb{R}^{k \times n}$ es la matriz de pesos y $b_1 \in \mathbb{R}^k$ el *bias*, de modo que f_1 va de $\mathbb{R}^n \rightarrow \mathbb{R}^k$.

Ahora, consideremos una segunda capa $f_2 : \mathbb{R}^k \rightarrow \mathbb{R}^m$, con $W_2 \in \mathbb{R}^{m \times k}$ y $b_2 \in \mathbb{R}^m$:

$$f_2(y) = W_2y + b_2.$$

Al componer ambas sin una función de activación de por medio:

$$f_2(f_1(x)) = W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2).$$

El producto W_2W_1 es de nuevo una matriz $W' = W_2W_1 \in \mathbb{R}^{m \times n}$. De igual forma, b_1 es multiplicado por W_2 , que lo lleva de $\mathbb{R}^k$ a $\mathbb{R}^m$, y al sumarle b_2 obtenemos otro vector $W_2b_1 + b_2 = b' \in \mathbb{R}^m$. Por lo tanto:

$$f_2(f_1(x)) = W'x + b',$$

que tiene la misma forma que una única capa $f' : \mathbb{R}^n \rightarrow \mathbb{R}^m$. Repitiendo el argumento, cualquier número de capas colapsa en una sola $W'x + b'$, por lo que la red tiene la misma capacidad expresiva que un modelo lineal de una sola capa.

- b. Considere una neurona con dos entradas $x_1 = 3$ y $x_2 = -2$, pesos $w_1 = 0,4$ y $w_2 = 0,6$, y sesgo $b = -0,1$. Calcule la preactivación z y la activación $a = \sigma(z)$. Interprete el resultado, ¿Qué le indica este valor sobre la predicción del modelo?

$$z = w_1x_1 + w_2x_2 + b = (0,4)(3) + (0,6)(-2) + (-0,1) = -0,1.$$

$$a = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{0,1}} \approx 0,4750.$$

La activación $a \approx 0,475$ se interpreta como la probabilidad estimada de que la entrada pertenezca a la clase positiva, es decir, $P(y = 1 | x) \approx 0,475$. Como este valor está apenas por debajo de 0,5, el modelo predice la clase negativa, pero por un margen muy pequeño. Esto nos indica que es un *sample* que realmente no es muy parecido a alguna de las clases del conjunto de entrenamiento y el modelo no “está muy seguro” de la predicción realizada.

Task 2

Responda de forma clara y deje evidencia de sus cálculos:

- a. Muestre paso a paso como se obtiene la fórmula de entropía cruzada binaria a partir del principio de máxima verosimilitud bajo una distribución de Bernoulli. Indique claramente en que paso se aplica el logaritmo y por qué eso es válido.

Para una clasificación binaria, se cuenta con labels $y \in \{0, 1\}$ y predicciones probabilísticas $\hat{y} \in (0, 1)$. La distribución de Bernoulli nos indica lo siguiente:

$$P(y | \hat{y}) = \hat{y}^y (1 - \hat{y})^{1-y}$$

Es decir, la probabilidad es $\hat{y}$ si $y = 1$ y $1 - \hat{y}$ si $y = 0$. Para un conjunto de N muestras independientes, la función de *likelihood* (L) se define como el producto de probabilidades de cada observación individual dado que para dos eventos independientes $P(A \cap B) = P(A) \times P(B)$:

$$L = \prod_{i=1}^N P(y_i | \hat{y}_i) = \prod_{i=1}^N \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$$

Entonces, el principio de *maximum likelihood* busca maximizar esta función. Ahora, por conveniencia (poder trabajar con sumatorias, evitar underflow, gradiente más manejable, etc.) podemos aplicar la función de logaritmo natural a cada una de las probabilidades. Dado que el logaritmo es una función estrictamente creciente, los parámetros que maximizan L son exactamente los mismos que maximizan $\log L$. Entonces, tenemos que:

$$\log L = \log \left(\prod_{i=1}^N \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i} \right)$$

Dado que $\log(ab) = \log a + \log b$ y $\log(a^b) = b \log a$ podemos reescribir $\log L$ como una sumatoria:

$$\log L = \sum_{i=1}^N (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

Ahora, como la convención es minimizar una función de pérdida simplemente podemos multiplicar la función por -1 y los parámetros que la minimizan son los mismos que maximizan

la función original. Adicionalmente, buscamos el error promedio por sample y por lo tanto dividimos el resultado dentro de N . Resultando la función de *binary cross-entropy*:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

- b. Para un mismo ejemplo de entrenamiento con etiqueta $y = 0$, calcule la pérdida $\mathcal{L}$ cuando el modelo predice $\hat{y} = 0,1$, $\hat{y} = 0,5$ y $\hat{y} = 0,9$. ¿Qué observa sobre el comportamiento de la pérdida a medida que la predicción se aleja de la etiqueta correcta? ¿Es ese comportamiento deseable en un sistema de clasificación?

Dado que la etiqueta real es $y = 0$, la fórmula general de la pérdida es la siguiente:

$$\mathcal{L} = -\log(1 - \hat{y})$$

Calculando las pérdidas:

- Para $\hat{y} = 0,1$: $\mathcal{L} = -\log(1 - 0,1) = -\log(0,9) \approx 0,105$
- Para $\hat{y} = 0,5$: $\mathcal{L} = -\log(1 - 0,5) = -\log(0,5) \approx 0,693$
- Para $\hat{y} = 0,9$: $\mathcal{L} = -\log(1 - 0,9) = -\log(0,1) \approx 2,303$

A medida que la predicción $\hat{y}$ se aleja de la etiqueta correcta $y = 0$, el valor de la pérdida crece de manera drástica y no lineal. Esto significa que la función penaliza con mucha mayor severidad a las predicciones que están más alejadas de la realidad. Esta dinámica coincide con lo descrito en un artículo consultado [1], donde se señala que este efecto de penalización es una de las razones para integrar logaritmos en la función de pérdida.

Este comportamiento es altamente deseable en un sistema de clasificación. Durante el entrenamiento, nos interesa castigar de forma estricta al modelo cuando genera predicciones *confidently wrong*. A nivel de optimización, la prioridad es corregir rápidamente los casos donde la red está más confundida y segura de su error (como predecir $\hat{y} = 0,9$ cuando $y = 0$), obligándola a ajustar sus parámetros más drásticamente debido a la magnitud del error.

- c. Un ingeniero propone usar error cuadrático medio en lugar de entropía cruzada para entrenar la regresión logística. Formule un argumento técnico concreto contra esa decisión.

Nota: Tuvimos una pregunta similar en un laboratorio en el curso de Inteligencia Artificial, la cita hace referencia hacia mi propio informe de laboratorio por temas de brevedad.

El uso del *Mean Squared Error* (MSE) en regresión logística presenta un problema de optimización. Dado que la función sigmoide introduce una ecuación trascendental, el MSE carece de una solución de forma cerrada y nos vemos obligados a utilizar métodos iterativos como el descenso de gradiente [2].

Para que el descenso de gradiente garantice encontrar la solución óptima, requiere operar sobre una función convexa. Al aplicar MSE sobre las salidas de una función sigmoide tenemos una superficie de pérdida no convexa, introduciendo múltiples mínimos locales y puntos silla donde la gradiente desvanece [2]. Esto causa que el optimizador se quede atascado. Por el contrario, la entropía cruzada mantiene es estrictamente convexa en tareas de clasificación, garantizando la convergencia hacia el único mínimo global.

Adicionalmente, las gradientes resultantes al utilizar MSE son sumamente pequeñas. Esto hace que el optimizador sea aún más propenso a atascarse en algún punto relativamente plano sobre la superficie.

Task 3

a. Dado el grafo de cómputo de la regresión logística:

$$x \rightarrow z = w^T x + b \rightarrow \hat{y} = \sigma(z) \rightarrow \mathcal{L}(\hat{y}, y)$$

Derive $\frac{\partial \mathcal{L}}{\partial w}$ paso a paso usando la regla de la cadena. Identifique en que paso aparece el resultado $\hat{y} - y$ y explique por que ese resultado es matemáticamente conveniente para el entrenamiento.

El grafo de cómputo tiene tres pasos: $z = w^T x + b$, $\hat{y} = \sigma(z)$ y $\mathcal{L}(\hat{y}, y)$, donde $\mathcal{L}$ es la entropía cruzada binaria:

$$\mathcal{L}(\hat{y}, y) = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})).$$

Como w solo afecta a $\mathcal{L}$ de forma indirecta (primero a través de z , y z a su vez a través de $\hat{y}$), la regla de la cadena nos dice que basta con multiplicar las derivadas parciales de cada paso del grafo:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}.$$

Derivando la entropía cruzada respecto a $\hat{y}$:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}.$$

Y derivando la sigmoide respecto a z :

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y}).$$

Al multiplicar ambas, el término $\hat{y}(1 - \hat{y})$ se cancela y queda simplemente:

$$\frac{\partial \mathcal{L}}{\partial z} = \hat{y} - y.$$

Aquí es donde aparece el resultado $\hat{y} - y$. Falta un último paso, $\frac{\partial z}{\partial w} = x$ (directo de $z = w^T x + b$), así que el gradiente completo es:

$$\frac{\partial \mathcal{L}}{\partial w} = (\hat{y} - y) x.$$

Este resultado es matemáticamente conveniente por dos razones principales. Primero, evita el problema del *vanishing gradient*, donde el gradiente se vuelve tan pequeño que el optimizador no logra actualizar los pesos para mejorar el modelo. Normalmente, la función sigmoide causa este problema porque su derivada es muy pequeña (0.25 como máximo, tendiendo a 0 en los extremos). Cabe destacar que esta cancelación beneficiosa ocurre únicamente en la capa de salida; al utilizar varias capas ocultas con activación sigmoide, el desvanecimiento del gradiente reaparece [3]. Segundo, calcular el gradiente resultante se vuelve computacionalmente barato y es directamente proporcional al error.

- b. Utilizando el script de Python proporcionado como recurso complementario de la semana, ejecute el entrenamiento y reporte: el valor del costo J en la época 0, en la época 250 y en la época final. Grafique la curva de costo en función de las épocas. Incluya la gráfica como imagen en su entrega e interprete brevemente lo que muestra sobre la convergencia del modelo.

- Época 0: $J = 0,693147$
- Época 250: $J = 0,275725$
- Época final: $J = 0,248261$

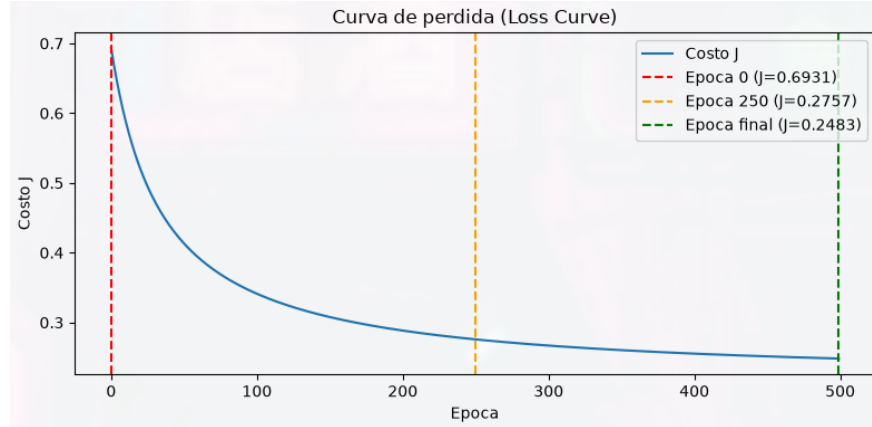


Figura 1: Curva de costo J en función de las épocas.

La curva muestra una caída pronunciada durante las primeras ~ 50 -100 épocas, pasando de $J \approx 0,69$ a valores cercanos a 0,3. A partir de ahí la pérdida sigue disminuyendo pero de forma cada vez más lenta, aplanándose conforme se acerca a la época final ($J \approx 0,248$). Este comportamiento es característico de la convergencia del descenso de gradiente. Al inicio los pesos están lejos del óptimo (prácticamente predicciones aleatorias) y el gradiente es grande, por lo que las actualizaciones son grandes. A medida que el modelo se acerca a un mínimo el gradiente se reduce y las actualizaciones se vuelven más pequeñas. Que la curva sea estrictamente decreciente y no oscile sugiere que la tasa de aprendizaje utilizada es adecuada para este problema. En conclusión, la gráfica de la curva de pérdida evidencia que el modelo logró converger de manera adecuada.

Uso de IA: Se utilizó un LLM para generar la gráfica utilizando Matplotlib, luego de haber modificado el código para que la pérdida de cada época se fuera registrando se utilizó el siguiente prompt:

“I’ve got the training of an ML model using gradient descent saved in the costs array, index indicates epoch and value at index indicates loss. I need to graph the loss curve as well as specific points in epochs 0, 250 and 500.”

El prompt funciona porque se le dió una explicación detallada al LLM de lo que tenía que llevar a cabo. Además, se le dió una tarea bastante delimitada al proporcionarle el array anterior a la instrucción.

Referencias

- [1] Daniel Godoy. *Understanding binary cross-entropy / log loss: a visual explanation*. Towards Data Science. 2018. URL: <https://towardsdatascience.com/understanding-binary-cross-entropy-log-loss-a-visual-explanation-a3ac6025181a/>.
- [2] José Antonio Mérida Castejón. *Laboratorio 2*. Informe de laboratorio. Inteligencia Artificial - CC3085, Universidad del Valle de Guatemala, 2026. URL: <https://github.com/Jose-Merida-UVG/Artificial-Intelligence-CC3085/blob/main/Lab-02/Informe.pdf>.
- [3] José Antonio Mérida Castejón. *Laboratorio 4*. Informe de laboratorio. Inteligencia Artificial - CC3085, Universidad del Valle de Guatemala, 2026. URL: <https://github.com/Jose-Merida-UVG/Artificial-Intelligence-CC3085/blob/main/Lab-04/Informe.pdf>.